Content Guide

Working with NOLF2 models

Basic model creation

All of the models in NOLF2 were modeled, textured and animated in Maya 4.0. We currently have working exporters for Maya3.0/4.0, and 3dStudioMax3.0/4.0. All of the basics of model creation mentioned here are specific to Maya, but the concepts apply to Max-created models.

Models and Polycounts
Models should be created using polygons. There are limits on the number of polygons of a particular model depending on the function and necessary detail level of that model. In NOLF2, the character models ranged from 1700 to 2700 triangles. This is a good range for most characters, but the limit could go as high as 4-5000 triangles in isolated situations. In addition to the main models, NOLF2 used hand-built level-of-detail (LOD) models for most of the gameplay characters (and all of the multiplayer models). The rule of thumb used was to create two duplicate models and reduce on to 60% and the other to 25% of the original model's polycount. LODs are not necessary, but are an optimization that will improve rendering speed when many models are onscreen simultaneously. More will be mentioned later about setting up LODs for a model.

Textures and RenderStyles

Textures can be created using any digital painting tool and should have dimensions that are a power of 2 pixels in length (…64,128,256, 512…). Model textures are generally kept square, but rectangular shaped textures are ok as long as the dimensions fill this requirement. A Photoshop plugin is included to allow creation of Jupiter's DTX texture format. In addition, plugins for Maya and Max are included to allow the use of DTX textures (Maya: IMFLithDTX.dll, MAX: MaxDTXLoader40.bmi). A model can theoretically have up to 16 textures, but each part of the model using a different texture needs to be separated into a unique piece. A model can be made up of as many as 64 separate pieces, but keeping as few as possible is desirable to avoid slowing model rendering down. Most NOLF2 characters have only two pieces, a head and a body, for this reason. In addition to simple textures, each piece can have a unique "render style". There are a lot of these that were created for very specific purposes, but the ones most commonly used for models are: envmap.ltb (for adding an alpha masked environment map, like with super soldiers), glass.ltb (for creating gray-scale alpha masked transparency, like eyeglasses, and even Cate's eyelashes), transparent.ltb (for creating things like trees with alpha mask), furfins.ltb and furshell.ltb (basically another way of doing alpha masked transparency, used for things like Armstrong's beard). There isn't currently a way to duplicate the effects of these in Maya, but they're not too bad to set up by just checking them in the game. One model piece can have only one render style, and usually pieces are combined based on the way model's render styles are distributed.

Skeletons

After creating the model and textures, the model geometry is bound to a skeleton. In Maya, this consists of a hierarchy of one or more joints. All Maya models should be attached to skeletons using 'smooth bind' in the Skin menu. By default, the exporter will only export joints that are bound to geometry, or the parents of such joints. In some cases it is necessary have other joints in the hierarchy exported. This is done by creating a Boolean attribute called 'ForcedNode' on the joint and setting its value to 1. Every exported model must have at least one joint. Inanimate models, such as lamps, chairs, etc. will need only one joint. Most NOLF2 characters have 35. Again it's possible to use a lot of joints, but keeping the number as low as possible will improve performance. Model LODs can be made even more efficient by removing unneeded joints from the lower detail models. This can be done in Maya by using the 'Edit Smooth Skin > Remove Influence' function. A sample Maya4.0 scene has been provided to use as a starting point for further experimentation (Tools\Docs\Character_sample.mb). This file is exactly the same as the multiplayer version of the pilot model (\chars\models\multi\pilot.ltb). (Unfortunately, we have no sample MAX model scenes.)

Exporting models

To use the exporter plugin in Maya, place the file 'MayaModelExport40.mll' in the Maya bin\plugins\ folder and the file 'LithTechModelExportOptions.mel' in the Maya scripts\others\ folder. Make sure the plugin is loaded in Maya's Plugin Manager window.

The exporter plugin exports a text version of the Lithtech file format, '.lta'. It will also export a compressed version, '.ltc', to save disk space. The format should be specified in the filename supplied to the plugin. The exporter uses the command line format:

> lithtechModelExport –all [-usePlaybackRange <*bool*> -append <*bool*> -ignoreBindPose <bool> -exportWorldNode <*bool*> -scale <*float*>] <animation name> <file name>

The optional flags are defined as follows:

- **-usePlaybackRange** – setting this to '1' will export only the visible range of an animation. '0' will export all keyframes in the scene.
- **-append** – setting this to '1' will append the animation to an existing file, allowing models to contain many animations. Setting it to '0' (the default) will overwrite the file if it exists.
- **-ignoreBindPose** – This option determines whether the bind pose of a model or the first frame of an animation is used as a base for deformation. This was set to '1' for nearly all NOLF2 models because of the child model workflow described below.
- **-exportWorldNode** – This allows multiple skeletons in a scene to be exported as one model if set to '1'. It adds a 'world node' as a parent to all exported skeletons. It was generally set to '0' for NOLF2 models.
- **-scale** – this will scale a model by the specified float value. This can be helpful since it can be tricky to scale animations in Maya.
- **animation name** – the name of an animation. When appending animations, if there is an existing animation of the same name, it will need to be deleted in ModelEdit before it can be used.
- **file name** – this should contain the entire path to the desired file. The format should be either '.lta', or '.ltc".

The exporter will use these options to decide which skeleton(s) should be exported, and will include all visible geometry bound to it.

The concept of 'child models' was used for most characters in NOLF2. The child model is a model that is used solely to store animations (and animation specific data). When a child model is applied to another model, that model will share all the animation data from the child model. This allows continual edits to be made on models without needing to worry about re-exporting large numbers of animations. It also allows many different models to use the same animation set. The base character should be exported unanimated at its bind pose with the animation called 'base'. A child model is created by using the 'append' option in the exporter. Be very careful, since accidentally not setting the append flag will simply overwrite the file without warning. Also, when joint names are case-specific when appending and must match exactly the names of the model.

A Maya script that eases the export/model setup workflow described here was created by the NOLF2 animator. It creates an interface to save many of the settings described here and in the next section in a Maya scene. It was used for nearly all the models that were exported throughout production and is presented here as is… it worked for me - I hope it works for others. If nothing else it provides a starting point for others to understand how the lta format can be used. It is described mode fully in another section.

Model setup using ModelEdit

After exporting a character model, a few things need to be done to get it to appear in the game. These things are done in ModelEdit. This tool has been around for a few years and hasn't changed a lot, in spite of our workflow changing a lot. What that means is that there are a lot of options that probably haven't been used in a while and may be obsolete.

The new Lithtech file format exists in two forms. The exporter creates a user-friendly data file (lta or ltc), but this needs to be compiled as an ltb file for use in the engine. An lta file is a text file containing all the model and animation data. An ltc file is a compressed version of this, and is what was used for most NOLF2 models. ModelEdit only reads lta or ltc files, not ltb files. ModelEdit can be used to convert between ltc and lta files. In addition to this, a command line tool, LTC.exe, can be used to convert between lta and ltc files. For help using this tool, execute the command 'LTC'. ModelPacker.exe is used by ModelEdit to create ltb files.

There are several things that need to be set up in ModelEdit to get models working. As an example, a fully setup character, Game\Chars\Models\Character_Sample.lta, has been provided that is the exported version

of the Maya scene Character_Sample.mb.

User Dims

The most basic thing that needs to be set on models is the User Dims. This is basically a bounding box for each model. To see this box in ModelEdit, select 'Options > Show User Dimensions'. By default the dims are a long box. This needs to be changed to more closely match the size of the model. This can be done either by adjusting the '+' and '-' buttons in the 'Animation Edit' section of ModelEdit or by selecting 'Animation > Dimensions…' and entering appropriate values. One thing to know about user dims is that the X value must always equal the Z value. If not the engine will pick one and apply it to both, producing strange results. Another thing to notice is that user dims can be set per animation. To avoid possible timing issues with changing dims, they are usually set the same for all animations on a character. The only exception in NOLF2 is the crouching animations for player characters. All human characters in NOLF2 have X=24, Y=53, Z=24.

Weight Sets

In order to render in the engine, character models need to have several 'weight sets' created. Weight Sets are used by the programmers to tell a character to play more than one animation at a time, such as the recoil animations or the upper/lower body blending in multiplayer. Select 'Model > Edit Weighted Animation Blending' in ModelEdit. Weight Sets are created one at a time by clicking the 'Add Set' button and entering the name. The following sets need to be added to every character model: Null, Upper, Lower, blink, twitch. To set the Weight Sets, select one or more nodes in the 'Nodes' window and set the appropriate value in the 'Current Node Weight' window. (When setting this number, don't press enter, as it will exit the dialogue and frustrate you).

- **Twitch** - select all the nodes in the node window and set the Weight to '2'. Setting the weight to '2' tells the engine to additively blend animations and in this case allows the twitch on dead bodies and recoil on shot enemies.
- **Blink** – select the Eyelid node (if one exists) and set its value to 2. This will cause the engine to play an animation called 'blink' if it exists constantly as long as the character is alive and blend it over whatever the character is doing. This could be used creatively…
- **Upper** – select all the nodes in the upper body of the character and set the weight to '1'. This tells the engine to replace the animation on these nodes with the desired upper body animation.
- **Lower** – select all the nodes in the lower body of the character and set the weight to '1'. Note that if there is a node controlling the translation of the model, it should be in the lower body.
- **Null** – leave all nodes set to zero.

Note that other than the Upper and Lower sets in multiplayer characters, all of these can be left at zero and the models will work fine. The sets just need to exist for the engine to be able to set up each character. Non-character models do not need weight sets.

Sockets

Sockets are created to define locations where other models or FX can be attached to characters. The most noticeable place where sockets are used is in the gun hands of character models. This is the only socket I can think of that is necessary for a character to function properly. This socket must be named 'RightHand', but can be attached to any node in the model (for example, the laser shooting super soldier has a RightHand socket attached to his Head node.) To create this socket in ModelEdit, select the desired node (usually the right hand) in the Nodes window. Then select 'Sockets > Add Socket…', and enter the name 'RightHand'. The socket will be created at the location and orientation of it's parent node. Usually this will need to be edited. This is most easily done by showing attachment models in ModelEdit. (Unfortunately, new sockets don't show attachments until the model is saved and re-opened.) Select 'Options > Show Sockets' and 'Options > Show Attachments'. Now double click the socket name (RightHand) in the 'Sockets' window. In the 'Attachment' field of this properties window, enter or browse to the location of an attachment model lta file (gun, flashlight…). Also in this window are entry boxes for orientation, translation, and scale values. These are very helpful when you know exactly what they need to be, but can be a little counterintuitive when starting from scratch since they are based on the orientation of the parent node. It's easier to use the 'Transform Edit' controls. The Red, Green, and Blue boxes correspond to the colored axes of the socket in the ModelEdit viewport. Translation and Rotation are usually all that is needed to position the attachment into place. Scale is usually only used if an attachment that is shared among several characters needs to be adjusted to fit onto larger (or smaller) body part. All the sockets that you create are listed in the 'Sockets' window in ModelEdit. Other important sockets that every character should have are LeftFoot and RightFoot. These are needed to create footprints and accurately located footstep sounds. Other than this the main uses for sockets are hats, glasses, holstered weapons. To see how these and other sockets are set up, examine the sample character.

Most characters have about a dozen sockets even though many of them aren't used by all the characters. This is for consistency, so no one needs to remember which character gets which sockets. A socket name will not be valid unless it is appears on a list determined by game code. The full available list can be seen by opening DEdit, selecting an AI and opening the attachments property. One other thing… the exporter plug-in allows sockets to be created and exported from Maya. To do this create an object (usually a locator) and add a Boolean attribute called 'sockets' and set it to '1'. This object can be used to orient objects within Maya and may be easier to work with than ModelEdit's controls.

Importing Sockets and Weight Sets

Sockets and Weight Sets are two things that are commonly imported from similar existing models. This can be done by selecting 'File > Import…'. To import only Sockets and Weight Sets, check the boxes next to those options, and uncheck Animations. Then browse to a previously set up model with a matching skeleton and click 'Open'. This can save a lot of tedious work, but some sockets may still need to be adjusted depending on the situation. Among the other options, Animations can also be imported from one model to another (instead of using a child model). When importing Animations, you should always import User Dims and Translations simultaneously to avoid needing to reset this information. The 'UV Coords' is no longer used.

Child Models

Child models are added in ModelEdit by selecting 'Child Model > Add ChildModel…' and browsing to the desired lta or ltc file. The only requirement for a model to be a valid child model is that the hierarchy must match exactly that of the parent model. A model can have as many as 32 child models, some of which can be added in game code as needed. Generally, only 1-3 child models are needed, depending on the character, and they will be listed in the 'Child models' window in ModelEdit. If a child model has drastically different proportions than a character, you may notice the character distorting to match those proportions. This can be corrected by checking the box next to the distorted nodes in ModelEdit's Nodes window. This tells the engine to ignore the translation information from the child model for this node and use only the rotation. In most NOLF2 characters, all the nodes are checked except translation, movement, and most Face nodes. These nodes contain mostly translation animation, so ignoring translation will kill the animation. The Face nodes that should be checked are the eye nodes and the eyelid node, since they only contain rotation animation. Note that this only works if a character shares the exact face setup as the child model. If this is not the case, all the face nodes should be checked.

Texture Setup

Setting up textures and render styles in ModelEdit is done in the Piece Info window. This is accessed by highlighting a model piece in the 'Pieces' window and selecing 'Piece > Piece Info…'. This can be the most confusing part of setting up a model. The main things to understand here are the Texture Indices and Render Style Index. All of the textures and render styles associated with a model are listed in an attribute file. Characters are in modelbutes.txt and props are in proptypes.txt. Here is an example of a Super Soldier's texture list:

```
Skin0         = "chars\skins\SSLtBody.dtx"
Skin1         = "chars\skins\SSLtHead.dtx"
Skin2         = "chars\skins\SSLtPack.dtx"
Skin3         = "chars\skins\shinyspot.dtx"
RenderStyle0  = "RS\default.ltb"
RenderStyle1  = "RS\envmap.ltb"
RenderStyle2  = "RS\glass.ltb"
RenderStyle3  = "RS\Glow.ltb"
```

The Texture Index refers to the Skin number in this list. The Render Style Index refers to the Render Style number in this list. For example the 'body' piece of the model will use Skin0 and RenderStyle0, so the indices will stay at the default values of zero. Some render styles use multiple textures. The only commonly used render style like this is envmap.ltb. An example of its use would be the Super Soldier's backpack. This uses RenderStyle1, Skin2 as the main texture map, and Skin3 as the environment map. To set this up, the 'Number of Textures' option needs to be set to '2'. Then Texture Index 0 should be set to '2' (for Skin2) and Texture Index 1 should be set to '3' (for Skin3'). The Render Style Index should be set to '1' (for RenderStyle1). There is also an option for 'Render Priority'. This is used to insure proper sorting of transparent models. All non-transparent models should have this set to '0'. Most models using alpha transparency render styles should have this set to '1' to insure that that piece will always be rendered properly when it is in front of an opaque piece. Complex model setups, like Armstrong's beard, can have a wide range of render priorities to sort the pieces properly.

Saving and Compiling

The last thing that needs to happen in ModelEdit is to save and compile the model. To Save and Compile, select 'File > Save and Compile…'. There are several options here, but only a couple that are ever actually used. One is the compression type. The default compression type is (RLE 16) which in the case of some animations, is too much compression. I generally compress unanimated models at '(RLE 16)' and compress any thing with subtle animations using '(RLE16)PlayerView'. This difference is that with (RLE16)PlayerView, everything is compressed to 16bit except translation which stays at 32bits. This basically results in smoother animations. The other commonly used option is 'Exclude Model Geometry'. This can be used for any model that is exclusively a child model and allows for really good data compression since the geometry is simply thrown out. The good news about the compile window is that all the settings are saved per model, so the only need to be set when first exported, if at all.

Other ModelEdit functions

Here are the other most commonly used ModelEdit functions.

Animation Editing

ModelEdit has a number of functions to allow simple editing of animation data. An animation cannot be edited if it is being shared from a child model. In the 'Animations' window, the animations are listed with a check mark in the box next to the name if they can be edited. Right clicking on the name of an animation will bring up a menu with options to rename, duplicate, or delete the animation. The animation time slider displays the keyframes of an animation as red (or green) ticks. For editing purposes, ModelEdit allows you to 'tag' a group of keyframes. Keyframes can be tagged individually by double clicking the red ticks in the time slider. This actually toggles the tagged state, so double clicking again will untag the keyframe. Tagged keyframes will appear as blue ticks. Groups of keyframes can be tagged by holding down the 'Shift' key and click-dragging the mouse across the group of red ticks. The tagged state of keyframes is not saved with the file. Right clicking the time slider opens a menu. Here are the useful options in this menu:

- **Delete Keyframe** – This will delete the current keyframe from the animation. It will not work on the first or last frame of the animation.
- **Delete Tagged Keyframes** – This allows multiple keyframes to be tagged and deleted simultaneously. Deleting unnecessary keyframes is a useful file size optimization.
- **Clip After Keyframe** – This will delete all the keyframes after the current keyframe.
- **Clip Before Keyframe** – This will delete all the keyframes before the current keyframe.
- **Insert Time Delay** – This will allow a time delay between two keyframes to be specified, and is used a lot in tweaking the timing of animations in cut-scenes.

More useful functions can be found in the 'Animation' menu:

- **Set Animation Rate** – This changes the keyframe rate of an animation. Most NOLF2 animations come from motion capture data, which has 30 frames per second. This function is used to reduce that rate. Most NOLF2 AI animations are reduced to 7.5 frames per second, while cut-scene animations are 10 frames per second. Tagged frames are protected from this function, so if there are rapid motions in the animation, tagging those frames will allow them to retain the full rate.
- **Set Animation Length** – This will scale the speed of an animation. During NOLF2 production, it was the easiest way to accurately set the timing of animations.
- **Set Animation Interpolation** – Interpolation is what the engine does to blend from one animation to another. This function allows the time of this blend to be set. The default is 200 ms, and that is good for most purposes. Looping animations and cut-scene animations usually have 0 interpolation.
- **Make Continuous -** This will duplicate the first frame of an animation and place it at the end to create a rough looping effect. This isn't perfect in most cases, but is a quick way to get a looping animation for testing purposes.
- **Reverse** – This will reverse the frames of an animation so that it plays backwards.
- **Create Single Frame** – This creates a one frame animation from the current frame of an existing animation.
- **Translation** – This is used to add a positional offset to an animation.
- **Rotate** – This is used to add a rotational offset to an animation.

Frame Strings

It is also possible to add commands to animations so that they are executed at a desired keyframe. This is done in ModelEdit by setting the animation to the desired keyframe and entering the command in the 'Frame String' text field (at the bottom of the ModelEdit interface). A keyframe that has a Frame String will appear as a green tick in the animation timeslider, as opposed to the default red tick. Multiple commands can be added to the same keyframe by separating the commands with a semicolon. There are many possible commands that can be used here – pretty much any command that can be sent to a character through triggers in the game will also work here. The most common ones are:

- **FOOTSTEP_KEY 1** – Right Footstep. This will play the appropriate sound effect when the right foot hits the ground and should be placed that point in the animation. It will also leave a footprint at the correct place in the snow.
- **FOOTSTEP_KEY 2** – Left Footstep. This should be placed when the left foot hits the ground.
- **DOOR** – Activate a door object. This should be placed at the keyframe where the door should start opening.
- **OPEN** – This command is used with smart objects to activate an object associated with the smart object. An example of this is the 'Desk' smart object where the OPEN command activates a chair to move when the animation pulls it from under the desk. For more information on smart objects see the AI docs.
- **FX *<fx name>*** - This will play an FX object at the desired keyframe. An example of this in action is the dissolve body FX (FX AIBodyRemove).
- **Attach *<socket name> <attachment name>*** – This will cause an attachment to be spawned at the specified socket and keyframe. An example is the cigarette appearing in an AI's hand at the appropriate time (Attach LeftHand cigarette).
- **Detach *<socket name>*** – This will remove an attachment from a character at the desired keyframe.

Importing LODs

Model LODs are another confusing part of model setup. Since they are not necessary, they may not be worth the trouble when experimenting. Nevertheless, here is how they can be imported using ModelEdit. LOD models should first be exported as a separate lta file, and the pieces should be named exactly as the pieces in the main file. LODs are imported on a piece-by-piece basis from this file. In ModelEdit, highlight the desired piece in the 'Pieces' window and select 'File > Import Custom LODs…'. Browse to the file containing the LOD pieces and click 'Open'. A window will open containing a list of all the pieces in that file. Select the appropriate piece and click 'OK'. Now open the '+' sign next to the piece name in the Pieces window. It should open to two '0.00' children. Here's the tricky part that will be frustrating. One of these is the original and the other is the LOD, but it can be difficult to guess which one. Usually the top 0.00 piece is the LOD, so highlight it and select 'Piece > Piece Info…'. The Texture and Render Style information should match the original Piece. In the LOD Properties section, set the Distance to a value other than 0. This will be the distance where the original model piece will swap with the new LOD. This process will need to be repeated for each piece of each LOD. Most of the characters in NOLF2 have two full LODs, one at 250 units, and on at 550.

Command String

The model's Command String is used to store various commands and settings used by the engine when setting up a model. In previous versions of Jupiter, there were many commonly used commands that were placed here, but many of those are now obsolete. The only thing the Command String is currently being used for is enabling shadows on a model. To access the Command String in ModelEdit, select 'Model > Command String…'. To turn in shadow casting simply enter "ShadowEnable" in the text field.

Piece Merging

A piece-merging feature was added to help optimize the number of geometry pieces in a model. It will merge all pieces that share a render style and texture. To use it highlight the desired pieces in the 'Pieces' window and select 'Piece > Merge Selected'. This allows modelers to keep models in separate pieces in the modeling software if desired and wait to combine them in the exported model for optimization.

Null LODs

It is possible to have model pieces disappear when models are a certain distance from the player. This is done in ModelEdit by highlighting the desired piece in the 'Pieces' window and selecting 'Piece > Create Null LOD…', then enter the desired distance. This was used as an optimization in Armstrong's beard, which mostly disappears at about 600 units.

Bute Files

All models in the game need to be defined in an attribute file. Characters are defined in 'modelbutes.txt', props are defined in 'proptypes.txt', and attachments are defined in 'attachments.txt'. There is a not a lot that needs to be understood about these files in order to add to them, but not following the format exactly can cause disastrous results. For this reason, they should be modified at your own risk, and never without first backup them up. Most new models are created by simply cutting and pasting new model listings from existing, similar models and changing the number, Name, the model filename, the Skin and RenderStyle lines. It is very important that each model has a unique number and name, and that each attribute file contains no gaps in numbers (example: a file that contains [Model56] and [Model58], but not [Model57] will crash the game. Also, a file that contains two [Model56] listings will crash the game). This is pretty important, but it's really all that needs to be understood to add most new models. Characters can be a little more complex for two reasons. One is the Skeleton section, which is used to define hit detection per skeleton and can be enormously painful to deal with. For this reason, it is recommended that experimenting with new characters should stick to the existing sample skeleton (Character_Sample.mb). A lot can be done with it without any changes. The other extra piece of information for characters is the 'DeathmatchModels' section. To add new multiplayer models, in addition to the model listing already described, the model must be added to this list using the 'Name' of the model listing (example 'Name36 = "NewModelName"'). This covers all the needed basics for model setup in NOLF2, but again modify at your own risk and be sure to backup the originals.

LtExport.mel Script

As mentioned in the exporter section, as MEL script was created to help streamline our workflow and give the ability to store a lot of settings in the Maya scene that would otherwise need to be set in ModelEdit every time a model is re-exported. It is intended to be an interface to the exporter plugin. This is 100% artist created tool and most of it was created as a means to learn MEL scripting. Because of this, it is unsupported and may not be the best way to do everything… but it works for us, and nearly all NOLF2 (and TRON) models were exported using this script as an interface. The script requires the file 'LTC.exe' to support the compressed ltc version of the lta file format. To use this script, place it in a user script directory, or Maya's Scripts\Others\ folder. Enter the command 'ltExport' to open the interface. Included is a brief description of options, settings, and known limitations in the script.

Edit Options

The 'Edit Options' section contains functions that attempt to mimic ModelEdit functionality:

- UserDims – This button creates a bounding box that can be scaled normally to set the dims. It correctly locks the X and Z values to avoid bad dims. The scale values will be saved on the skeleton so that they can always be exported correctly.
- Sockets – This opens a window that will give options to create and delete sockets and works very much like the ModelEdit dialogue.
- Piece Info – For texture setup, this works exactly like ModelEdit's Piece info window. Select a Piece and press this button and the window will open to allow settings to be made. It also has a place to enter the correct game texture path so that textures will show up in ModelEdit automatically (This may not be usable except during production).
- CommandString – This is exactly like the ModelEdit Command String window. Anything entered here will be saved with the skeleton.
- ChildModels – This allows child models to be specified. A list of child models is saved with the skeleton.
- NodeFlags – This mimics the 'Nodes' window of ModelEdit, allowing the node check box setting to be saved with the skeleton.

Other Setting Options

Here are the other available setting options:

- Append Animation – When checked on, this will cause the exporter to always append animations without warning. When off, a warning will come up to prevent accidentally overwriting files. This sets the '-append' flag of the exporter flag of the exporter plugin.

- Use TimeSlider range – This sets the '-usePlaybackRange' flag of the exporter plugin.
- ExportWeightSets – This will create (but not correctly set) the default set of weight sets needed to for characters to work in the game.
- Export Child Models – This was added to allow a model to have child models in Maya, but export without them if needed.
- CompileModel – This will automatically compile the model as an ltb file. When checked on, an option is exposed to 'Compile with no geometry'. This is used when exporting many animations to a child model.
- Dims – This provides a numerical input for setting User Dims.
- Scale – This sets the '-scale' flag of the exporter plugin.
- Interpolation – This sets the interpolation time of an animation. It is the same as using 'Set Animation Interpolation' in ModelEdit.
- ToolPath – This should provide the folder where the file 'LTC.exe' exists
- GamePath – This may not be usable, but shouldn't affect the working of the script.

In addition to these options, the script will export LODs for models by looking for a very specific naming convention in the Maya scene. I can't think of a better way to describe this setup than to point to the sample Maya character setup scene (Character_Sample.mb). It contains LOD pieces that have been hidden to prevent them from exporting. Unhiding them and exporting this scene using the ltExport Script will result in fully set up LODs.

Limitations

There are lots of issues that were never addressed due to lack of time. Here are a couple that you'll probably notice:

- There are a lot of 'Browse' buttons in the interface. Most of them don't work at all and the main File Browser will only work if you select an existing lta or ltc file. I never did figure out how to get file browsing to work correctly.
- The script will give an error if it is run when there are no skeletons in the scene.
- The script recognizes the first skeletal hierarchy in the scene to be the export skeleton. This is always the top joint listed in the outliner or the leftmost one in the hypergraph. Occasionally a hidden skeleton could be listed before the desired one and cause a lot of confusion.
- There is a weird bug that occurs when child models are specified and exported using this script… the normals are destroyed. Fortunately this can be remedied in ModelEdit by selecting 'Model > Generate Vertex Normals'. Because of this, unless there are several child models, I don't really use the child model part of the script much.
- The 'Node Flags' option will be made invalid if the skeleton is changed after setting the flags. The exporter won't notice this, and may cause an assortment of problems if invalid values are exported.